

Day 2: Variables and Data Types

Goal:

Learn how to **declare variables** and use **different data types** in JavaScript.

1. What Are Variables?

Variables are containers that **store values** in your program. You can think of them as labeled boxes where each box holds a specific kind of data.

In JavaScript, we use:

- `var` (old, avoid using today)
 - `let` (recommended for values that **can change**)
 - `const` (recommended for values that **should not change**)
-

2. `var`, `let`, and `const`

Keyword	Scope	Can Reassign?	Can Redeclare?	Hoisted?
<code>var</code>	Function-scoped	☒ Yes	☒ Yes	☒ Yes (undefined)
<code>let</code>	Block-scoped	☒ Yes	☒ No	☒ No (TDZ)
<code>const</code>	Block-scoped	☒ No	☒ No	☒ No (TDZ)

◇ Examples:

```
var name = "Alice"; // avoid using var in modern code
let age = 25;
const country = "USA";
```

```
// ☒ Reassigning let
age = 26;
```

```
// ☒ Reassigning const - causes an error
country = "Canada"; // ☒ TypeError
```

3. JavaScript Data Types

JavaScript has two main categories:

➤ Primitive Types:

1. **String** – Text data ("Hello")
2. **Number** – Numeric values (42, 3.14)
3. **Boolean** – True or false (true, false)
4. **Undefined** – A variable that has been declared but not assigned a value
5. **Null** – A deliberate non-value
6. **Symbol** – Unique and immutable values (advanced)
7. **BigInt** – Large integers beyond Number limits

➤ Non-Primitive Types:

- **Objects, Arrays, Functions**

4. Examples of Each Type

```
// String
let name = "John";

// Number
let age = 30;
let score = 99.5;

// Boolean
let isActive = true;

// Undefined
let x; // declared but not assigned

// Null
let y = null; // intentionally empty

// Symbol
let id = Symbol("id");

// BigInt
let bigNum = 1234567890123456789012345678901234567890n;
```

5. Type Coercion

JavaScript sometimes **automatically converts** one data type to another. This is called **type coercion**.

➤ Examples:

```
console.log("5" + 2); // "52" (number 2 is coerced to string)
console.log("5" - 2); // 3 (string "5" is coerced to number)
console.log(true + 1); // 2 (true becomes 1)
```

6. typeof Operator

The `typeof` operator returns the **data type** of a value.

➤ Examples:

```
console.log(typeof "Hello");      // string
console.log(typeof 42);           // number
console.log(typeof true);         // boolean
console.log(typeof undefined);    // undefined
console.log(typeof null);         // object (quirk in JS)
console.log(typeof Symbol("id")); // symbol
console.log(typeof 123n);         // bigint
```

Practice

1. Declare variables for each primitive type.
2. Log their values and types using `console.log` and `typeof`.
3. Try reassigning a `const` and observe the error.
4. Try operations like `"5" + 1`, `"5" - 1`, `true + 2` and explain the output.

➤ Sample Practice Code:

```
let name = "Sarah";
let age = 22;
let isStudent = false;
let grade;
let middleName = null;
let uniqueID = Symbol("id123");
let bigValue = 12345678901234567890n;

console.log(typeof name);      // string
console.log(typeof age);       // number
console.log(typeof isStudent); // boolean
console.log(typeof grade);     // undefined
console.log(typeof middleName); // object
console.log(typeof uniqueID);  // symbol
console.log(typeof bigValue);  // bigint

const planet = "Earth";
// planet = "Mars"; // ✗ Uncaught TypeError: Assignment to constant variable.
```

Key Takeaways

- Use `let` for variables that can change, and `const` for fixed values.
 - Know and use all 7 primitive data types.
 - Be aware of **type coercion** — it's both powerful and dangerous.
 - Use `typeof` to debug variable types.
 - Avoid `var` in modern JavaScript.
-

Input/Output Conceptual JavaScript Questions tailored for focusing on variables, data types, type coercion, `typeof`

10 1. What is the difference between `undefined` and `null`?

Definition:

- **undefined**: Automatically assigned to a variable **that has been declared but not assigned a value**.
- **null**: A deliberate assignment that indicates the **absence of any value**.

Example:

```
let a;  
let b = null;  
  
console.log(typeof a); // undefined  
console.log(typeof b); // object (legacy behavior)
```

Explanation:

- Use `undefined` when a value is **not yet set**.
 - Use `null` to **explicitly reset or empty** a variable.
-

10 2. What are the key differences between `let`, `const`, and `var`?

Definition:

Keyword	Scope	Reassignment	Redeclaration	Hoisting
<code>var</code>	Function	☒ Yes	☒ Yes	☒ Hoisted (value: <code>undefined</code>)
<code>let</code>	Block	☒ Yes	☒ No	☒ Not initialized (TDZ)
<code>const</code>	Block	☒ No	☒ No	☒ Not initialized (TDZ)

Example:

```
function test() {  
  if (true) {  
    var x = 10;  
    let y = 20;  
    const z = 30;  
  }  
  console.log(x); // 10  
  console.log(y); // ReferenceError  
}
```

Explanation:

- Prefer `let` and `const` in modern JS.
 - `const` prevents both **reassignment and redeclaration**.
-

10 3. How does JavaScript handle type coercion during operations?

■ Definition:

Type coercion means JavaScript **automatically converts types** to perform an operation.

☑ Example:

```
console.log("5" + 1); // "51" (number → string)
console.log("5" - 1); // 4   (string → number)
console.log(true + 1); // 2   (boolean → number)
```

💡 Explanation:

- `+` triggers string coercion if any operand is a string.
 - `-`, `*`, `/` trigger numeric coercion.
-

10 4. What will `typeof null` return and why is it misleading?

☑ Answer:

```
typeof null; // "object"
```

💡 Explanation:

This is a **bug** in JavaScript's original implementation. `null` is **not an object**, but `typeof null` still returns `'object'` due to backward compatibility.

10 5. When should you use `BigInt`, and how is it different from `Number`?

■ Definition:

- `Number` supports values up to $2^{53} - 1$
- `BigInt` supports arbitrarily large integers

☑ Example:

```
const num = 9007199254740991; // max safe integer
const big = 9007199254740991n + 1n;

console.log(big); // 9007199254740992n
```

💡 Explanation:

Use `BigInt` when working with **very large integers**, such as in cryptography, precision calculations, etc.

10 6. What is the temporal dead zone (TDZ) in JavaScript?

■ Definition:

TDZ is the **time between variable declaration and initialization**, where accessing a `let` or `const` variable throws a **ReferenceError**.

☑ Example:

```
console.log(a); // ReferenceError
let a = 10;
```

💡 Explanation:

- The variable exists in memory, but can't be accessed until initialized.
 - Only applies to `let` and `const`, **not** `var`.
-

10 7. Explain Symbol: what is it and when is it used?

■ Definition:

A `Symbol` is a **primitive type** introduced in ES6, used to **create unique property keys**.

☑ Example:

```
const sym1 = Symbol("id");
const sym2 = Symbol("id");

console.log(sym1 === sym2); // false
```

💡 Explanation:

Symbols are often used to create **private-like** or **unique** object properties to prevent name collisions.

10 8. How does hoisting work for `var`, `let`, and `const`?

■ Definition:

Hoisting means variable and function declarations are **moved to the top** of their scope during the compile phase.

☑ Example:

```
console.log(a); // undefined
var a = 10;

console.log(b); // ReferenceError
```

```
let b = 20;
```

💡 Explanation:

- `var` is hoisted **and initialized to undefined**
 - `let` and `const` are hoisted but **not initialized** (TDZ)
-

10 9. How to check the real type of a variable beyond `typeof`?

▣ Definition:

For objects, `typeof` is not always reliable. Use `Object.prototype.toString.call()`.

☑ Example:

```
console.log(typeof []); // "object"
console.log(Object.prototype.toString.call([])); // "[object Array]"

console.log(typeof null); // "object"
console.log(Object.prototype.toString.call(null)); // "[object Null]"
```

💡 Explanation:

Use this pattern for accurate type-checking of arrays, dates, null, etc.

10 10. Can a `const` variable be an object or array that's modified later?

☑ Answer:

Yes, but only the **binding is constant**, not the **contents**.

☑ Example:

```
const arr = [1, 2, 3];
arr.push(4); // allowed
arr = [5, 6]; // ✗ TypeError
```

💡 Explanation:

- `const` prevents reassignment of the **variable itself**, not the internal **mutability** of the object/array it references.
-

☑ Example 1: Displaying Data Types with `typeof`

```
<!DOCTYPE html>
<html>
```

```

<head>
  <title>Data Types with typeof</title>
</head>
<body>
  <h2>Check the Data Type</h2>
  <p id="result"></p>

  <script>
    let name = "Alice";
    let age = 30;
    let isStudent = false;
    let value;
    let nothing = null;

    let output = `
      name: ${typeof name}<br>
      age: ${typeof age}<br>
      isStudent: ${typeof isStudent}<br>
      value: ${typeof value}<br>
      nothing: ${typeof nothing}
    `;

    document.getElementById("result").innerHTML = output;
  </script>
</body>
</html>

```

☑ Example 2: Type Coercion in Action

```

<!DOCTYPE html>
<html>
<head>
  <title>Type Coercion Demo</title>
</head>
<body>
  <h2>Type Coercion Examples</h2>
  <p id="coercion"></p>

  <script>
    let result1 = "5" + 1;    // string + number = string
    let result2 = "5" - 1;    // string - number = number
    let result3 = true + 2;   // boolean + number = number

    let output = `
      "5" + 1 = ${result1}<br>
      "5" - 1 = ${result2}<br>
      true + 2 = ${result3}
    `;

    document.getElementById("coercion").innerHTML = output;
  </script>
</body>
</html>

```

☑ Example 3: let vs const Behavior


```

<!DOCTYPE html>
<html>
<head>
  <title>let vs const</title>
</head>
<body>
  <h2>Variable Behavior</h2>
  <p id="variableResult"></p>

  <script>
    let count = 5;
    const greeting = "Hello";

    count = 10;           // ☒ allowed
    // greeting = "Hi"; // ☒ Uncommenting this will cause an error

    document.getElementById("variableResult").innerHTML = `
      Updated count: ${count}<br>
      Greeting: ${greeting}
    `;
  </script>
</body>
</html>

```

☒ Example 4: User Input and Type Checking

```

<!DOCTYPE html>
<html>
<head>
  <title>User Input Type Check</title>
</head>
<body>
  <h2>Enter a Value</h2>
  <input id="userInput" type="text" placeholder="Type something..." />
  <button onclick="checkType()">Check Type</button>
  <p id="typeOutput"></p>

  <script>
    function checkType() {
      let input = document.getElementById("userInput").value;
      let type = typeof input;

      document.getElementById("typeOutput").innerHTML = `
        Value: "${input}"<br>
        Type: ${type}
      `;
    }
  </script>
</body>
</html>

```

☒ Example 5: Symbol and BigInt

```

<!DOCTYPE html>
<html>
<head>

```

```

    <title>Symbol and BigInt</title>
</head>
<body>
    <h2>Symbol and BigInt Demo</h2>
    <p id="advanced"></p>

    <script>
        const id = Symbol("userId");
        const big = 1234567890123456789012345678901234567890n;

        document.getElementById("advanced").innerHTML = `
            Symbol value: ${id.toString()}<br>
            Type of Symbol: ${typeof id}<br>
            BigInt value: ${big}<br>
            Type of BigInt: ${typeof big}
        `;
    </script>
</body>
</html>

```

1. Form Validation (String, Boolean, Coercion)

Scenario:

Check if all fields are filled before submitting a form.

```

<!DOCTYPE html>
<html>
<head><title>Form Validation</title></head>
<body>
    <h2>Registration Form</h2>
    <form onsubmit="return validateForm()">
        Name: <input type="text" id="name"><br><br>
        Age: <input type="text" id="age"><br><br>
        <button type="submit">Submit</button>
    </form>
    <p id="error" style="color:red;"></p>

    <script>
        function validateForm() {
            const name = document.getElementById("name").value.trim();
            const age = document.getElementById("age").value.trim();

            if (!name || isNaN(age) || Number(age) <= 0) {
                document.getElementById("error").innerText = "Please enter a valid name"
                return false;
            }

            return true;
        }
    </script>
</body>
</html>

```

2. Feature Flags using const, boolean, and Conditional

Logic

🔍 Scenario:

Enable/disable UI elements based on environment configuration.

```
<!DOCTYPE html>
<html>
<head><title>Feature Toggle</title></head>
<body>
  <h2>New Feature Preview</h2>
  <div id="featureBlock"></div>

  <script>
    const isFeatureEnabled = false;

    if (isFeatureEnabled) {
      document.getElementById("featureBlock").innerHTML = "<p>🚀 New Feature En
    } else {
      document.getElementById("featureBlock").innerHTML = "<p>🔒 Feature not av
    }
  </script>
</body>
</html>
```

☑ 3. Currency Conversion (Number, Type Coercion)

🔍 Scenario:

Convert user input from one currency to another using exchange rate.

```
<!DOCTYPE html>
<html>
<head><title>Currency Converter</title></head>
<body>
  <h2>USD to INR</h2>
  Amount in USD: <input id="usd" type="text">
  <button onclick="convert()">Convert</button>
  <p id="result"></p>

  <script>
    const exchangeRate = 83.3; // USD to INR

    function convert() {
      const usd = document.getElementById("usd").value;
      const inr = Number(usd) * exchangeRate;

      if (isNaN(inr)) {
        document.getElementById("result").innerText = "Please enter a valid num
      } else {
        document.getElementById("result").innerText = `₹${inr.toFixed(2)}`;
      }
    }
  </script>
</body>
```

</html>

☑ 4. Local Storage with `typeof` and `JSON` (Object, String)

🔍 Scenario:

Save and retrieve a user object from `localStorage`.

```
<!DOCTYPE html>
<html>
<head><title>Local Storage Demo</title></head>
<body>
  <h2>Save User</h2>
  <button onclick="saveUser()">Save User</button>
  <button onclick="loadUser()">Load User</button>
  <p id="userInfo"></p>

  <script>
    const user = {
      name: "Alice",
      age: 28,
      loggedIn: true
    };

    function saveUser() {
      localStorage.setItem("user", JSON.stringify(user));
      alert("User saved!");
    }

    function loadUser() {
      const stored = localStorage.getItem("user");
      const parsedUser = JSON.parse(stored);

      document.getElementById("userInfo").innerText = `
        Name: ${parsedUser.name}, Age: ${parsedUser.age}, Logged In: ${parsedUser.loggedIn}
      `;
    }
  </script>
</body>
</html>
```

💡 Dev Insight:

- Objects must be serialized (`JSON.stringify`) and parsed (`JSON.parse`).
- `typeof parsedUser` → `"object"`

☑ 5. Theme Toggle (Boolean, `let/const`, DOM interaction)

🔍 Scenario:

Allow user to toggle between light and dark theme.

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>Theme Toggle</title>
  <style>
    body.dark {
      background-color: #222;
      color: white;
    }
  </style>
</head>
<body>
  <button onclick="toggleTheme()">Toggle Theme</button>

  <script>
    let isDark = false;

    function toggleTheme() {
      isDark = !isDark;
      document.body.className = isDark ? "dark" : "";
    }
  </script>
</body>
</html>

```

💡 Dev Insight:

- A common use case in modern apps.
- Boolean state stored in `let` so it can be toggled.

📦 Summary of Covered Concepts in Real Life

Concept	Real Life Usage
<code>let, const</code>	State management, feature toggles, DOM updates
Data Types	Form inputs, localStorage, server response parsing
Type Coercion	Input handling, validation, arithmetic with strings
<code>typeof</code>	Debugging, conditional branching
Objects	User profile, configs, request payloads

🔑 ACTIVITY DETAILS

**1 Task: Declare and Log All Data Types **

Instructions:

Create variables for each of these data types:

- String
- Number
- Boolean
- Null

- Undefined
- Object
- Symbol
- BigInt

Code Sample:

```
let name = "Alice";           // String
let age = 30;                 // Number
let isActive = true;         // Boolean
let emptyValue = null;       // Null
let notDefined;              // Undefined
let user = { name: "John" }; // Object
let uniqueId = Symbol("id");  // Symbol
let largeNumber = 12345678901234567890n; // BigInt

console.log(typeof name);
console.log(typeof age);
console.log(typeof isActive);
console.log(typeof emptyValue); // 'object' (JS quirk)
console.log(typeof notDefined);
console.log(typeof user);
console.log(typeof uniqueId);
console.log(typeof largeNumber);
```

2 Task: Apply `typeof` in a Debugging Scenario

Instructions:

Create a function that accepts any value and logs its type. Test with different inputs.

Code Sample:

```
function logType(value) {
  console.log(`Value: ${value}, Type: ${typeof value}`);
}

logType("123");
logType(123);
logType(true);
logType(null);
logType(undefined);
logType(123n);
```

**3 Task: Type Coercion Playground **

Instructions:

Demonstrate 3 examples each of:

- Implicit coercion
- Explicit coercion

Sure! Here are clear definitions for both:

Implicit Coercion

Definition:

Implicit coercion (also called *type coercion*) is when JavaScript automatically converts a value from one data type to another during an operation without the programmer explicitly telling it to do so.

Explanation:

This happens behind the scenes, usually in expressions involving different types. For example, when you use the `+` operator between a number and a string, JavaScript converts the number to a string and concatenates them.

Example:

```
console.log('5' + 10); // Output: "510"
// The number 10 is implicitly converted to a string "10" and concatenated.
```

Explicit Coercion

Definition:

Explicit coercion is when the programmer manually converts a value from one data type to another using built-in functions or methods.

Explanation:

You clearly tell JavaScript to convert a value to a specific type using functions like `Number()`, `String()`, `Boolean()`, or methods like `.toString()`.

Example:

```
console.log(Number('5') + 10); // Output: 15
// The string '5' is explicitly converted to number 5 before addition.
```

Code Sample:

```
// Implicit
console.log("5" + 2);      // "52"
console.log("10" - 2);     // 8
console.log(true + 1);     // 2

// Explicit
console.log(Number("123")); // 123
console.log(Boolean(""));   // false
console.log(String(100));   // "100"
```

4 Task: Use `let`, `const`, and `var` to Show Scope

Instructions:

Build a function with a block scope and declare variables inside using `var`, `let`, and `const`. Log their behavior.

Code Sample:

```
function testScope() {
  if (true) {
    var a = 1;
    let b = 2;
    const c = 3;
  }

  console.log(a); // 1
  // console.log(b); // ReferenceError
  // console.log(c); // ReferenceError
}
testScope();
```

Also show reassignment:

```
let x = 10;
x = 20;

const y = 30;
// y = 40; // ✖ Error
```

5 Task: Mini Project – User Input Type Analyzer

Goal:

Build a mini app in HTML + JavaScript where a user enters a value and sees:

- The value
 - Its type
 - What happens when you try to add 10 to it (coercion test)
-

☒ Requirements:

1. Input field
 2. Button
 3. Output area to show:
 - Original value
 - `typeof`
 - Value + 10
 - Type after addition
-

Solution:

```
<!DOCTYPE html>
<html>
<head>
  <title>Type Analyzer</title>
</head>
<body>
```



```
<h2>🔍 JavaScript Type Analyzer</h2>
<input id="inputVal" placeholder="Enter something..." />
<button onclick="analyze()">Analyze</button>
<pre id="output"></pre>

<script>
  function analyze() {
    let input = document.getElementById("inputVal").value;
    let typeBefore = typeof input;
    let addedValue = input + 10;
    let typeAfter = typeof addedValue;

    let result = `
Original Value: ${input}
Type Before: ${typeBefore}
Value + 10: ${addedValue}
Type After: ${typeAfter}
`;
    document.getElementById("output").innerText = result;
  }
</script>
</body>
</html>
```